

JusticeFlow

AI-Powered Police Case & Evidence Management System

Database Schema — Complete Technical Reference for Team Use

FAST-NU Karachi | BCS-4D

DBMS · Software Design & Architecture · Artificial Intelligence · Operating Systems

Project Team

Muhammad Furqan Sheikh

Muhammad Abu Bakar

Abdullah Muaz Abbassi

Project Overview

JusticeFlow is a Pakistan-based, AI-powered Police Case and Evidence Management System designed as a multi-course integrated academic project. It spans four concurrent courses — Database Management Systems, Software Design and Architecture, Artificial Intelligence, and Operating Systems — with the explicit constraint that removing any single layer breaks critical system functionality.

The database is built on PostgreSQL and is organized into three schemas, seven SQL files, 34 tables, 18 stored functions, 28 triggers, and 62 performance indexes. Every design decision reflects real-world domain knowledge.

Technology Stack

Layer	Technology	Purpose
Database	PostgreSQL	Core relational engine across 3 schemas
Backend Application	C++	System calls, IPC, POSIX APIs
AI / ML	Python (scikit-learn, NumPy, SHAP)	Three ML agents for analysis
OS Integration	Linux POSIX APIs	Job scheduling, process management
Version Control	Git	Source and schema management

Three-Schema Architecture

The database is deliberately split into three schemas with strict isolation between them:

Schema	Purpose	Who Writes	Who Reads
public	All core operational tables — Officers, Cases, Evidence, Arrests etc.	justice_app (application)	justice_app, justice_ai
audit	Append-only legal audit trail — permanent and immutable	Triggers only (SECURITY DEFINER)	justice_app only
analytics	AI agent output — hotspot predictions, priority scores, workload suggestions	justice_ai (INSERT only)	justice_app (dashboard)

This isolation ensures that AI agents can never tamper with operational data, that audit records can never be modified by anyone, and that the application layer serves as the only bridge between the three schemas.

File 1 — 00_schema.sql

Purpose

This is the entry point for the entire database setup. It creates the three PostgreSQL schemas that house all tables, functions, and indexes. Without this file, none of the subsequent files can execute because they reference schema-qualified names such as `audit.Audit_Log` and `analytics.Crime_Hotspots`.

What It Contains

- `CREATE SCHEMA IF NOT EXISTS public` — the default schema for all operational tables
- `CREATE SCHEMA IF NOT EXISTS audit` — the append-only legal audit schema
- `CREATE SCHEMA IF NOT EXISTS analytics` — the AI agent output schema

Why IF NOT EXISTS

The `IF NOT EXISTS` clause makes the file safely re-runnable. If the schemas already exist (from a previous run or migration), PostgreSQL silently skips creation instead of throwing an error. This is a best practice for database initialization scripts.

Key Design Decision

The `public` schema already exists in every PostgreSQL database by default. Explicitly creating it here with `IF NOT EXISTS` documents intent and ensures the script is fully self-contained regardless of database state.

Execution Order

This file must always be run first. Every other file depends on these schemas existing before they can create objects inside them.

File 2 — 01_types.sql

Purpose

This file defines all custom ENUM types used across the database. ENUMs are PostgreSQL's way of constraining a column to a fixed set of valid string values — similar to a dropdown in a form. They must be created before any table references them, which is why this file runs second.

What is an ENUM?

An ENUM type stores only the values you explicitly define. Any attempt to insert a value not in the list is rejected at the database level before it reaches any application code. This provides data integrity without writing any CHECK constraints.

```
CREATE TYPE officer_rank_enum AS ENUM (  
    'CONSTABLE', 'HEAD_CONSTABLE', 'ASI', 'SI',  
    'INSPECTOR', 'DSP', 'SP', 'SSP', 'DIG', 'ADDL_IG', 'IGP'  
);
```

Complete Enum Inventory

Enum Name	Values	Used By
station_type_enum	ZONE_HQ, DISTRICT_HQ, DIVISION_HQ, POLICE_STATION, SUB_STATION	Stations
officer_rank_enum	CONSTABLE through IGP (11 ranks, BPS 7-22)	Officers, validation triggers
officer_status_enum	ACTIVE, SUSPENDED, ON_LEAVE, RETIRED, TERMINATED	Officers
case_officer_role_enum	DUTY_INCHARGE, SIO, IO, LEAD_INVESTIGATOR, SUPPORTING, EVIDENCE_CUSTODIAN	Case_Officers
gender_enum	MALE, FEMALE, OTHER	Persons
case_type_enum	20 criminal + 10 civilian types (30 total)	Cases, analytics
case_status_enum	REGISTERED, UNDER_INVESTIGATION, EVIDENCE_COLLECTED, PENDING_TRIAL, CLOSED, REOPENED	Cases, logs

Enum Name	Values	Used By
approval_status_enum	NOT_REQUIRED, PENDING_APPROVAL, APPROVED, REJECTED	Cases
evidence_type_enum	PHYSICAL, DIGITAL, TESTIMONIAL, FORENSIC, DOCUMENTARY	Evidence
evidence_status_enum	RECEIVED, SEALED, SENT_TO_LAB, RETURNED_FROM_LAB, PRODUCED_IN_COURT, DISPOSED	Evidence
custody_status_enum	IN_CUSTODY, BAIL_GRANTED, REMANDED, RELEASED, ESCAPED	Arrests
warrant_type_enum	ARREST, SEARCH, SEIZURE	Warrants
warrant_status_enum	ISSUED, EXECUTED, CANCELLED, EXPIRED	Warrants
bail_type_enum	REGULAR, ANTICIPATORY, INTERIM, SURETY	Bail_Records
bail_status_enum	ACTIVE, REVOKED, EXPIRED, CANCELLED	Bail_Records
forensic_request_status_enum	REQUESTED through CONTESTED (6 states)	Forensic_Lab_Requests
examination_purpose_enum	DNA, Fingerprint, Ballistics, Toxicology, Digital, etc.	Forensic_Lab_Requests
charge_sheet_status_enum	DRAFT, FILED, SUBMITTED_TO_COURT, ACCEPTED/REJECTED_BY_COURT	charge_sheets
sheet_type_enum	ORIGINAL, SUPPLEMENTARY	charge_sheets
vehicle_role_enum	STOLEN, USED_IN_CRIME, ABANDONED, EVIDENCE, SUSPECTS/VICTIMS_VEHICLE	Vehicle_Cases
seizure_status_enum	SEIZED, NOT_SEIZED, RELEASED, AUCTIONED, DESTROYED, RETAINED_FOR_EVIDENCE	vehicles
vehicle_type_enum	CAR, MOTORCYCLE, TRUCK, BUS, VAN, SUV, RICKSHAW, OTHER	vehicles
shift_type_enum	MORNING, AFTERNOON, NIGHT, SPLIT, ON_CALL	Duty_Roster

Enum Name	Values	Used By
duty_status_enum	SCHEDULED, ON_DUTY, COMPLETED, ABSENT, ON_LEAVE, SUSPENDED	Duty_Roster
audit_action_enum	INSERT, UPDATE, DELETE	audit.Audit_Log
audited_table_enum	8 table names (lowercase)	audit.Audit_Log
hotspot_risk_level_enum	LOW, MEDIUM, HIGH, CRITICAL	analytics.Crime_Hotspots
priority_level_enum	LOW, MEDIUM, HIGH, CRITICAL	analytics.Case_Priority_Scores
assignment_status_enum	SUGGESTED, ACCEPTED, REJECTED, AUTO_EXPIRED	analytics.Officer_Workload_Assignments

Pakistani Police Rank Hierarchy

The officer_rank_enum directly mirrors the real Pakistani police rank structure with their corresponding BPS (Basic Pay Scale) grades:

Rank	BPS Scale	Belt Prefix	Typical Role
CONSTABLE	BPS-07	PC-XXXX	Frontline patrol
HEAD_CONSTABLE	BPS-09	HC-XXXX	Senior constable
ASI	BPS-11	K-XXXX	Assistant Sub Inspector
SI	BPS-14	K-XXXX	Sub Inspector — FIR filing
INSPECTOR	BPS-16	K-XXXX	SHO — Station House Officer
DSP	BPS-17	K-XXXX	Deputy Superintendent
SP	BPS-18	K-XXXX	Superintendent
SSP	BPS-19	K-XXXX	Senior Superintendent
DIG	BPS-20	K-XXXX	Deputy Inspector General
ADDL_IG	BPS-21	K-XXXX	Additional Inspector General
IGP	BPS-22	K-XXXX	Inspector General of Police

File 3 — 02_tables.sql

Purpose

This is the largest and most important file. It defines all 30+ tables across all three schemas. Tables are the structural backbone of the system — every piece of data the application reads or writes lives in one of these tables.

Table Creation Order

PostgreSQL requires that referenced tables exist before a FOREIGN KEY can point to them. The file therefore creates tables in a strict dependency order:

Order	Table Group	Tables
1	Foundation	Stations, Persons
2	Officers layer	Officers, Officer_Rank_History, Officer_Deployments
3	Core case entity	Cases, Case_Officers
4	Case roles	Complainants, Victims, Witnesses, Accused, Accused_Associations
5	Case history logs	Case_Status_Log, Case_Jurisdiction_History
6	Evidence chain	Evidence, Evidence_Custody_Log
7	Legal actions	Warrants, Arrests, Bail_Records
8	Investigation tools	Forensic_Lab_Requests, Forensic_Request_Evidence
9	Court process	charge_sheets, charge_sheet_accused
10	Vehicles	vehicles, Vehicle_Cases
11	Duty management	Patrol_Routes, Duty_Roster
12	Audit schema	audit.Audit_Log
13	Analytics schema	Crime_Hotspots, Case_Priority_Scores, Officer_Workload_Assignments, Model_Performance_Log

Key Design Patterns

1. Persons as Master Identity Table

All human entities — Officers, Complainants, Witnesses, Victims, Accused — have their identity stored in a single Persons table keyed by CNIC (Computerized National Identity Card). This prevents duplication and allows one person to play multiple roles across different cases.

```
Persons (cnic PK)
├── Officers (cnic FK)      ← a person who is a police officer
├── Complainants (person_cnic FK) ← filed a complaint
├── Witnesses (person_cnic FK)  ← gave testimony
├── Victims (person_cnic FK)   ← was harmed
└── Accused (person_cnic FK)  ← suspected/charged
```

2. Soft Delete on Evidence

Evidence records can never be physically deleted from the database. The Evidence table has an `is_deleted` boolean column. Triggers actively block any DELETE statement and raise an exception. Only soft deletes (UPDATE SET `is_deleted` = TRUE) are permitted, which preserves the legal chain of custody permanently.

3. Auto-Generated Document Numbers

Every major legal document has a structured unique identifier generated automatically by triggers. No application code needs to manage numbering:

Document	Format	Example
FIR	FIR-YYYY-STATIONCODE-NNNN	FIR-2024-KHD-0001
Evidence	EVD-{FIR_NUMBER}-NNNN	EVD-FIR-2024-KHD-0001-0003
Arrest	ARR-YYYY-STATIONCODE-NNNN	ARR-2024-KHD-0001
Warrant	WRT-YYYY-STATIONCODE-NNNN	WRT-2024-KHD-0001
Bail	BAIL-YYYY-STATIONCODE-NNNN	BAIL-2024-KHD-0001
Forensic Request	FLR-YYYY-STATIONCODE-NNNN	FLR-2024-KHD-0001
Charge Sheet	CS-YYYY-STATIONCODE-NNNN	CS-2024-KHD-0001
Duty	DUTY-YYYY-STATIONCODE-NNNN	DUTY-2024-KHD-0001

4. Sequence Registry for Collision-Free Numbering

A dedicated `Sequence_Registry` table ensures that no two documents ever receive the same number, even if two officers file paperwork simultaneously. The `get_next_seq()` function uses an atomic ON CONFLICT DO UPDATE upsert to increment a counter and return the new value in a single database operation, eliminating any race condition.

5. CHECK Constraints Embedded in Table Definitions

Business rules are enforced at the database level through CHECK constraints. This means the rules apply regardless of what application or script accesses the database:

- Retirement date must be set when and only when status is RETIRED
- Belt number format must match rank (PC- for Constable, HC- for Head Constable, K- for all others)
- BPS scale must be one of the 11 valid values (7,9,11,14,16,17,18,19,20,21,22)
- Evidence soft delete requires deletion reason and timestamp
- Arrest release requires a release reason
- Withdrawal of a complainant requires a withdrawal reason
- Charge sheet `laws_invoked` array cannot be empty once status moves past DRAFT

6. Analytics Schema Isolation

The analytics schema tables intentionally have no FOREIGN KEY constraints pointing to the public schema. This is a deliberate architectural decision. AI agents write predictions to analytics tables, and if a case or officer were deleted from public tables, orphaned FK violations would block the analytics INSERT. The isolation means analytics always writes successfully, and referential integrity is maintained by the application layer instead.

File 4 — 03_functions.sql

Purpose

This file contains all 18 stored functions (PL/pgSQL procedures) that implement the system's business logic inside the database engine. Functions are attached to triggers or called directly by the OS job scheduler. Writing logic here rather than in application code means it executes faster (inside the database process), applies consistently regardless of which client connects, and is version-controlled with the schema.

Six Function Categories

Category 1 — Number Generators (8 functions)

One function per document type. Each generator looks up the relevant station code, reads the current year, calls `get_next_seq()` for an atomic sequential number, then assembles the formatted string and assigns it to the NEW row before it is written to disk.

Atomicity Guarantee

The `get_next_seq()` helper uses `ON CONFLICT DO UPDATE` — a PostgreSQL atomic upsert. Two simultaneous inserts are serialized at the row level inside `Sequence_Registry`. No two documents can ever receive the same number.

Category 2 — Expiry Functions (3 functions)

These functions are not attached to triggers. They are registered with the OS layer (cron-style scheduler) and called on a schedule:

Function	Schedule	Action
<code>expire_warrants()</code>	Daily at 00:01	Sets all ISSUED warrants past <code>valid_until</code> to EXPIRED
<code>expire_bail_records()</code>	Daily at 00:01	Sets all ACTIVE bail records past <code>valid_until</code> to EXPIRED
<code>expire_workload_assignments()</code>	Hourly	Sets SUGGESTED AI assignments past <code>expires_at</code> to AUTO_EXPIRED

Each expiry function uses `GET DIAGNOSTICS` to count affected rows and emits a `RAISE NOTICE` log line, giving the OS scheduler a readable audit trail of every nightly run.

Category 3 — Validation Functions (6 functions)

These fire BEFORE INSERT or UPDATE and abort the operation with a descriptive RAISE EXCEPTION if a business rule is violated:

Function	Rule Enforced
validate_fir_filing_rank()	Only ASI, SI, or INSPECTOR can file an FIR
validate_warrant_request_rank()	Only INSPECTOR or above can request a warrant
validate_case_closure_rank()	Only INSPECTOR (SHO) or above can approve case closure
validate_officer_duty_status()	Officers who are SUSPENDED, ON_LEAVE, or ABSENT cannot be assigned to cases
validate_officer_case_limit()	No officer can hold more than 10 active cases simultaneously
validate_patrol_route_station()	A patrol route can only be assigned to an officer at the same station

Category 4 — Immutability Functions (4 functions)

These functions exist solely to raise exceptions, permanently blocking destructive operations on legal records:

- prevent_evidence_hard_delete() — blocks any DELETE on the Evidence table
- prevent_audit_log_delete() — blocks any DELETE on audit.Audit_Log
- prevent_audit_log_update() — blocks any UPDATE on audit.Audit_Log
- prevent_locked_charge_sheet_update() — blocks any UPDATE on a charge sheet once is_locked = TRUE

Category 5 — State Sync Functions (4 functions)

These automatically keep related records in sync so application code never has to manually maintain derived state:

Function	Triggered By	Effect
sync_evidence_sent_to_lab()	INSERT on Forensic_Request_Evidence	Marks linked evidence as SENT_TO_LAB
sync_evidence_returned_from_lab()	UPDATE on Forensic_Lab_Requests	Marks all linked evidence as RETURNED_FROM_LAB when report is delivered
auto_lock_charge_sheet()	UPDATE on charge_sheets	Sets is_locked=TRUE automatically on SUBMITTED_TO_COURT transition
log_case_status_change()	UPDATE on Cases	Writes a row to Case_Status_Log on every status change

Category 6 — Audit Function (1 function)

`audit.log_change()` is the most important function in the system. It is a single reusable function attached to all 8 audited tables. Every INSERT, UPDATE, or DELETE on those tables automatically produces an immutable audit record capturing:

- Which table and which record (by primary key)
- The full before and after row snapshots as JSONB
- The database session user (e.g. `justice_app`)
- The application-level officer ID and belt number (from session variable `app.current_officer_id`)
- The OS process ID (`pg_backend_pid()`) and client IP address
- The exact timestamp

The function is declared SECURITY DEFINER, meaning it runs with the privileges of its owner (the superuser who created it) rather than the caller. This allows it to INSERT into `audit.Audit_Log` even though the calling user (`justice_app`) has no write access to the audit schema.

File 5 — 04_triggers.sql

Purpose

Triggers are the wiring that connects functions to tables. This file contains no logic itself — it simply attaches each function from `03_functions.sql` to the correct table at the correct timing (BEFORE or AFTER) for the correct operation (INSERT, UPDATE, or DELETE). There are 28 triggers in total.

BEFORE vs AFTER Triggers

Timing	When it Fires	What it Can Do	Used For
BEFORE	Before row is written to disk	Modify NEW values, cancel the operation by raising exception	Number generators, validations, immutability, auto-lock
AFTER	After row is successfully written	Cannot modify the row, can INSERT into other tables	Audit logging, state sync, status logging

Trigger Execution Order

When a Case is INSERTed, PostgreSQL fires triggers in this sequence:

```
INSERT INTO Cases (...)  
→ BEFORE: trg_generate_fir_number      (assigns FIR number)  
→ BEFORE: trg_validate_fir_filing_rank (checks officer rank)  
→ Row written to disk  
→ AFTER:  trg_log_case_status_change    (writes to Case_Status_Log)  
→ AFTER:  trg_audit_cases                (writes to audit.Audit_Log)
```

PostgreSQL fires multiple BEFORE triggers in alphabetical order by trigger name, then multiple AFTER triggers in alphabetical order. All trigger names are prefixed with trg_ to keep this ordering clean and predictable.

Complete Trigger Reference

Number Generator Triggers (8)

Trigger	Table	Timing	Event
trg_generate_fir_number	Cases	BEFORE	INSERT
trg_generate_evidence_number	Evidence	BEFORE	INSERT
trg_generate_arrest_number	Arrests	BEFORE	INSERT
trg_generate_warrant_number	Warrants	BEFORE	INSERT
trg_generate_bail_number	Bail_Records	BEFORE	INSERT
trg_generate_forensic_request_number	Forensic_Lab_Requests	BEFORE	INSERT
trg_generate_charge_sheet_number	charge_sheets	BEFORE	INSERT
trg_generate_duty_number	Duty_Roster	BEFORE	INSERT

Validation Triggers (6)

Trigger	Table	Timing	Event
trg_validate_fir_filing_rank	Cases	BEFORE	INSERT
trg_validate_warrant_request_rank	Warrants	BEFORE	INSERT
trg_validate_case_closure_rank	Cases	BEFORE	UPDATE
trg_validate_officer_duty_status	Case_Officers	BEFORE	INSERT
trg_validate_officer_case_limit	Case_Officers	BEFORE	INSERT
trg_validate_patrol_route_station	Duty_Roster	BEFORE	INSERT / UPDATE

Immutability Triggers (4)

Trigger	Table	Blocks
trg_prevent_evidence_hard_delete	Evidence	All DELETE operations
trg_prevent_audit_log_delete	audit.Audit_Log	All DELETE operations
trg_prevent_audit_log_update	audit.Audit_Log	All UPDATE operations
trg_prevent_locked_charge_sheet_update	charge_sheets	UPDATE when is_locked = TRUE

State Sync Triggers (4)

Trigger	Table	Timing	Event
trg_sync_evidence_sent_to_lab	Forensic_Request_Evidence	AFTER	INSERT
trg_sync_evidence_returned_from_lab	Forensic_Lab_Requests	AFTER	UPDATE
trg_auto_lock_charge_sheet	charge_sheets	BEFORE	UPDATE
trg_log_case_status_change	Cases	AFTER	UPDATE

Audit Triggers (8 tables)

One AFTER INSERT OR UPDATE OR DELETE trigger on each of the 8 audited tables, all calling audit.log_change():

Trigger	Table
trg_audit_cases	Cases
trg_audit_evidence	Evidence
trg_audit_officers	Officers
trg_audit_arrests	Arrests
trg_audit_warrants	Warrants
trg_audit_charge_sheets	charge_sheets
trg_audit_bail_records	Bail_Records
trg_audit_accused	Accused

File 6 — 05_indexes.sql

Purpose

Indexes are separate data structures that PostgreSQL maintains alongside tables. They allow the database engine to find rows matching a WHERE clause without scanning every row in the table — similar to an index at the back of a textbook. Without indexes, every query on a large table requires a full sequential scan, which becomes catastrophically slow as data grows.

Three Types of Indexes Used

1. Regular Indexes

Index a full column. Used for simple equality lookups and foreign key joins:

```
CREATE INDEX idx_cases_station_id ON Cases (station_id);
```

2. Composite Indexes

Index two or more columns together. Used when queries always filter by multiple columns simultaneously. The column order matters — the index is only useful when the query starts with the leftmost column:

```
-- Dashboard filter: always filters by station then status
CREATE INDEX idx_cases_station_status ON Cases (station_id, case_status);
```

3. Partial Indexes

Index only a subset of rows matching a WHERE condition. Much smaller and faster than a full index because it skips irrelevant rows:

```
-- Only index active (non-relieved) case officer assignments
CREATE INDEX idx_case_officers_active
ON Case_Officers (officer_id, case_id)
WHERE relieved_at IS NULL;
```

Index Summary by Schema

Section	Tables Covered	Index Count	Notable Indexes
1	Stations	3	station_code, district, parent_station_id (partial)
2	Officers	5	cnic, station_id, current_rank, status, composite station+rank
3	Cases	12	fir_number, status, type, filed_at, lat+lon spatial (partial)
4	Case Role Tables	9	Complainants, Victims, Witnesses, Accused — case_id and person_cnic each
5	Evidence	8	case_id, type, status, active-only partial, custody log
6	Arrests & Warrants	8	accused_cnic, case_id, status, expiry partial on warrants
7	Bail Records	3	arrest_id, status, expiry partial
8	Forensic Lab	3	case_id, status, junction evidence_id
9	Charge Sheets	4	case_id, status, filed_by, accused_cnic
10	Vehicles	5	registration_number, seizure_status, owner_cnic (partial)
11	Duty & Patrol	7	officer_id, station_id, duty_date, today composite
12	audit schema	4	table+record composite, officer_id, changed_at, action
13	analytics schema	9	risk_level, priority_score, assignment_status, expiry partial

Indexes Designed for Specific System Components

System Component	Index It Relies On
AI Hotspot Analyzer (DBSCAN)	idx_cases_lat_lon — spatial coordinate pair
AI Priority Recommender (Random Forest)	idx_cases_status_filed, idx_evidence_active
AI Workload Balancer (Hungarian)	idx_officers_station_rank, idx_case_officers_active
Nightly warrant expiry job (OS)	idx_warrants_expiry_check — partial on ISSUED only
Nightly bail expiry job (OS)	idx_bail_records_expiry_check — partial on ACTIVE only
Hourly AI suggestion expiry (OS)	idx_workload_expiry_check — partial on SUGGESTED only
Officer duty status check trigger	idx_duty_roster_today — composite officer+date+status
Audit trail queries	idx_audit_log_table_record, idx_audit_log_changed_at

File 7 — 06_permissions.sql

Purpose

This file implements role-based access control at the database level. Two application roles are created with carefully scoped permissions. No matter what the application code tries to do, the database engine will reject any operation that the role is not permitted to perform.

Two Roles

Role	Description	Connects From
justice_app	The main application role. Used by the C++ backend for all operational queries.	C++ application process
justice_ai	The AI agent role. Used by Python ML scripts to read public data and write predictions.	Python ML scripts

Permission Matrix

Schema	justice_app	justice_ai
public	SELECT, INSERT, UPDATE, DELETE on all tables	SELECT only on all tables
audit	SELECT on Audit_Log only	NO ACCESS — completely blocked
analytics	SELECT on all tables	INSERT only — cannot UPDATE or DELETE

Why These Restrictions Matter

justice_ai Cannot Write to public

AI agents read operational data to build their models and generate predictions. If they could write to public tables, a bug in a Python script could corrupt live case data. The separation guarantees that AI output is always contained in the analytics schema.

justice_ai Cannot Touch audit

Audit records are legal evidence. If AI scripts could read or write audit logs, they could either expose sensitive change history or tamper with the permanent record. The REVOKE ALL ensures zero access — not even SELECT.

justice_ai Can Only INSERT to analytics

AI agents should never be able to modify or delete their own past predictions. Past predictions are historical records needed for model performance tracking and explainability. INSERT-only access enforces immutability of AI output from the database level.

audit.Audit_Log Written by Triggers Only

Neither justice_app nor justice_ai can INSERT into audit.Audit_Log directly. The only writer is the audit.log_change() trigger function, which is declared SECURITY DEFINER and runs as its privileged owner. This means audit records can only be created by actual database operations — they cannot be fabricated by application code.

Security Architecture Summary

public schema: justice_app writes, justice_ai reads

audit schema: triggers write (SECURITY DEFINER), justice_app reads

analytics schema: justice_ai inserts, justice_app reads

No user can directly write to audit.

No AI process can modify public data.

No AI process can erase its own predictions.

Communication Assessment — The 7Cs

This section evaluates the JusticeFlow database schema documentation and design against the 7Cs of effective communication: Clear, Concise, Concrete, Correct, Coherent, Complete, and Courteous.

1. Clear

Clarity means the message is easy to understand with no ambiguity about intent or meaning.

Every SQL file in JusticeFlow is named with a numeric prefix and a descriptive name that immediately communicates its role: 00_schema, 01_types, 02_tables, 03_functions, 04_triggers, 05_indexes, 06_permissions. The ordering is not arbitrary — it encodes the execution dependency chain. A new team member reading the file list alone understands that schemas must exist before types, types before tables, and so on.

Inside each file, section headers use the same double-comment divider style (====), and every major block is preceded by a plain-English comment explaining what it does and why. Trigger names are prefixed with `trg_`, index names with `idx_`, and constraint names with `chk_` so their purpose is immediately visible in any database inspection tool without needing to read the definition.

The ENUM type names are self-documenting — `officer_rank_enum`, `case_status_enum`, `evidence_type_enum` — rather than opaque codes or integers that would require a lookup table to interpret.

2. Concise

Conciseness means delivering maximum information with minimum redundancy.

The single `audit.log_change()` function serves all 8 audited tables rather than duplicating logic into 8 separate functions. One function, one place to maintain, zero repetition. Similarly, the `get_next_seq()` helper is written once and called by all 8 number generator functions, eliminating 8 copies of the same atomic increment logic.

CHECK constraints are written inline with their table definitions rather than added as separate ALTER TABLE statements, keeping related logic co-located. Partial indexes with WHERE clauses replace larger full-table indexes, reducing storage and maintenance overhead while being more targeted in what they cover.

The Persons master table eliminates the need to store name, CNIC, and contact information in Complainants, Witnesses, Victims, and Accused separately — one canonical record per person, referenced by all role tables.

3. Concrete

Concrete communication uses specific, tangible details rather than vague generalities.

Every design decision in JusticeFlow traces back to a concrete real-world requirement. FIR filing is restricted to ASI, SI, and INSPECTOR ranks because those are the three ranks authorized under Pakistani CrPC 1898 to register First Information Reports. The maximum of 10 active cases per officer is not an arbitrary limit — it reflects the operational capacity discussed during the CPO interview.

Document number formats — FIR-YYYY-STATIONCODE-NNNN — match the actual format used in Pakistani police stations as confirmed through domain research. Belt number prefixes (PC- for Constable, HC- for Head Constable, K- for ASI and above) mirror the real physical belt numbers worn by officers in KPK and Sindh police.

Legal references in case types (PPC, CrPC 1898, PECA 2016, CNSA 1997, Arms Ordinance 1965, Zainab Alert Act 2020) are the actual Pakistani statutes that apply to those crime categories — not fictional placeholders.

4. Correct

Correctness means the information is accurate, the syntax is valid, and the logic is sound.

All five SQL files were reviewed and corrected across multiple iteration cycles before being run against a live PostgreSQL instance. Common errors identified and fixed included a duplicate enum definition for `approval_status_enum`, a wrong column type (`BIGINT` instead of `VARCHAR(15)`) on a CNIC reference, an inverted boolean condition in a CHECK constraint on charge sheet law arrays, a trailing comma inside an ENUM definition, and incorrect comma placement inside CHECK expressions.

The trigger timing decisions are architecturally correct: BEFORE triggers modify NEW values (number generators, auto-lock) while AFTER triggers write derived records (audit log, status log). Attempting to INSERT into another table from a BEFORE trigger is possible but risky — JusticeFlow correctly uses AFTER for all cross-table writes.

The SECURITY DEFINER declaration on `audit.log_change()` is the correct PostgreSQL mechanism for allowing a restricted user to indirectly write to a schema they have no direct access to — analogous to a `setuid` binary in Unix.

5. Coherent

Coherence means all parts fit together logically and support a unified whole.

JusticeFlow's four-course integration is coherent by design. The DBMS layer provides the structured data store. The OS layer provides the job scheduler that calls `expire_warrants()`, `expire_bail_records()`, and `expire_workload_assignments()` on a cron schedule. The AI layer reads from public schema via `justice_ai` and writes predictions to analytics schema. The SDA layer provides the C++ application that sits between the database and the user, setting the `app.current_officer_id` session variable that the audit function uses to identify the responsible officer.

Remove any one layer and something breaks. Without the OS scheduler, warrants never expire. Without the AI schema, the dashboard has no priority scores or hotspot alerts. Without the audit triggers, there is no legal chain of custody. Without the C++ application, there is no session context for the audit function to read. The coherence is structural, not cosmetic.

Within the schema itself, the three-schema split is coherent: public contains what happened, audit records who changed what and when, and analytics contains what the AI predicts will happen next. Each schema answers a different question.

6. Complete

Completeness means nothing essential is missing and the receiver has everything needed to act.

The schema covers the full lifecycle of a police case from initial complaint registration through FIR filing, investigation, evidence collection, forensic analysis, arrest, warrant issuance, bail, charge sheet preparation, and court submission. Every transition point has a corresponding log table (`Case_Status_Log`, `Case_Jurisdiction_History`, `Evidence_Custody_Log`, `Officer_Rank_History`).

The permissions file is complete in that it not only grants the necessary access but also explicitly REVOKES default PUBLIC access from the audit and analytics schemas — a step many implementations overlook. Without the REVOKE, any database user would inherit read access by default through the PUBLIC role.

The functions file is complete in that it includes not just the happy-path generators but also the expiry functions, the validation guards, the immutability blockers, the state sync functions, and the audit function — covering both normal operations and edge cases. The indexes file covers all three schemas including the audit and analytics tables that are often forgotten in index planning.

7. Courteous

Courtesy means the communication respects the audience and acknowledges their context and needs.

This documentation was written with two audiences in mind: teammates who need to understand the schema quickly to contribute to their respective course layers, and an instructor who needs to verify that the design decisions are academically sound and technically justified.

Every non-obvious decision is accompanied by a comment explaining why, not just what. The INHERITS approach for Officers was considered and deliberately rejected — documented in design discussion rather than silently avoided. The decision to use no foreign keys in the analytics schema is marked with an explicit comment in the table definition so no future reader mistakes it for an oversight.

The naming conventions are consistent throughout: tables use PascalCase, columns use snake_case, constraints use the chk_ prefix, indexes use idx_, triggers use trg_, functions use descriptive verb_noun patterns. This consistency respects the reader's time — once the pattern is learned, every name in the codebase is predictable.

Domain terminology — FIR, SHO, SIO, IO, CrPC, BPS scale, belt number — is used correctly as a Pakistani law enforcement professional would use it, showing respect for the domain and for the CPO whose interview informed these decisions.

7Cs Summary

- Clear — Self-documenting names, numbered files, plain-English comments
- Concise — Single audit function, shared helpers, no duplicated logic
- Concrete — Real Pakistani ranks, real formats, real legal references
- Correct — Verified against live PostgreSQL, reviewed across iterations
- Coherent — Four-course integration is structural, each layer depends on others
- Complete — Full case lifecycle, all schemas, all edge cases covered
- Courteous — Two-audience documentation, consistent conventions, justified decisions